

Contents

1. Contest	1
2. Data Structures	2
3. Graph	11
4. Number Theory	17
5. Strings	20
6. Theory	21

1. Contest

1.1. rng.h

```
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());

int random(int a, int b) {
    if (a > b)
        return 0;
    return a + rng() % (b - a + 1);
}

double random_double(double a, double b) {
    return a + (b - a) * (rng() / (double)rng.max());
}
```

1.2. template.h

```
#include <bits/stdc++.h>

#define tc \
    int t; \
    cin >> t; \
    while (t--) \
        solve();

using namespace std;
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
```

```
template <class T, class U = null_type, class chash = hash<T>>
using hset = gp_hash_table<T, U, chash>;
template <class T, class U = null_type, class cmp = less<T>>
using oset = tree<T, U, cmp, rb_tree_tag,
tree_order_statistics_node_update>;
#define MOD 1000000007
#define int long long
#define pa pair<int, int>
#define fr(i, a, b) for (int i = (a); i < (b); i++)
#define fnr(i, a, b) for (int i = (a); i >= (b); i--)
#define vi vector<int>
#define vpi vector<pa>
#define vvi vector<vi>
#define pb push_back
#define all(s) s.begin(), s.end()
#define set_bits \
    __builtin_popcountll // tells the number of
setbits or number of 1s
#define zero_bef __builtin_clzll // number of leading zeroes
#define sz(a) (int)a.size()
#define print(v) \
    for (auto printi : v) \
        cout << printi << ' '
// make a custom tempelate

using T = long long; // Generic type for templates
using ldbl = long double;

void solve() {}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    return 0;
}
```

2. Data Structures

2.1. FenwickTree.h

```
template <typename T>
class FenwickTree {
public:
    int n;
    vector<T> bit;
    // 1 based fenwick tree, bit[1] = a[1]
    FenwickTree(int size) : n(size), bit(n + 1, T{}) {}

    FenwickTree(const vector<T>& arr) :
    FenwickTree((int)arr.size() - 1) {
        build(arr);
    }

    void build(const vector<T>& arr) {
        bit.assign(n + 1, T{});
        for (int i = 1; i <= n; ++i) {
            bit[i] += arr[i];
            int parent = i + (i & -i);
            if (parent <= n) bit[parent] += bit[i];
        }
    }

    void update(int idx, T delta) {
        for (; idx <= n; idx += idx & -idx)
            bit[idx] += delta;
    }

    T query(int idx) const {
        T result{};
        for (; idx > 0; idx -= idx & -idx)
            result += bit[idx];
        return result;
    }
}
```

```
T query(int left, int right) const {
    return query(right) - query(left - 1);
}
};

template <typename T>
class FenwickTree2D {
public:
    int n, m;
    vector<vector<T>> bit;

    // 1 based 2D Fenwick tree. bit[i][j] covers the rectangle
    // ending at
    // (i, j) with dimensions (i & -i) by (j & -j).
    FenwickTree2D(int rows, int columns)
        : n(rows), m(columns), bit(n + 1, vector<T>(m + 1, T{}))
    {}

    FenwickTree2D(const vector<vector<T>>& arr)
        : FenwickTree2D((int)arr.size() - 1, (int)arr[0].size() -
1) {
        build(arr);
    }

    // how to use ?
    // write in code
    // FenwickTree2D<int>ft(#rows, #columns);
    // ft.build(arr);
    void build(const vector<vector<T>>& arr) {
        bit.assign(n + 1, vector<T>(m + 1, T{}));
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= m; ++j)
                update(i, j, arr[i][j]);
    }

    void update(int row, int column, T delta) {
```

```

    for (int i = row; i <= n; i += i & -i)
        for (int j = column; j <= m; j += j & -j)
            bit[i][j] += delta;
}

T prefix_sum(int row, int column) const {
    T result{};
    for (int i = row; i > 0; i -= i & -i)
        for (int j = column; j > 0; j -= j & -j)
            result += bit[i][j];
    return result;
}

T rectangle_sum(int top, int left, int bottom, int right)
const {
    return prefix_sum(bottom, right) - prefix_sum(top - 1,
right) -
        prefix_sum(bottom, left - 1) + prefix_sum(top - 1,
left - 1);
}
};

template <typename T>
class RangeFenwickTree {
public:
    int n;
    vector<T> bit1, bit2;

    RangeFenwickTree(int size) : n(size), bit1(n + 1, T{}),
bit2(n + 1, T{}) {}

    RangeFenwickTree(const vector<T>& arr) :
RangeFenwickTree((int)arr.size() - 1) {
        build(arr);
    }
};

```

```

void build(const vector<T>& arr) {
    bit1.assign(n + 1, T{});
    bit2.assign(n + 1, T{});
    for (int i = 1; i <= n; ++i)
        range_add(i, i, arr[i]);
}

void range_add(int left, int right, T value) {
    add(bit1, left, value);
    add(bit1, right + 1, -value);
    add(bit2, left, value * (left - 1));
    add(bit2, right + 1, -value * right);
}

T prefix_sum(int idx) const {
    return value(bit1, idx) * idx - value(bit2, idx);
}

T range_sum(int left, int right) const {
    return prefix_sum(right) - prefix_sum(left - 1);
}

private:
void add(vector<T>& bit, int idx, T value) {
    for (; idx <= n; idx += idx & -idx)
        bit[idx] += value;
}

T value(const vector<T>& bit, int idx) const {
    T result{};
    for (; idx > 0; idx -= idx & -idx)
        result += bit[idx];
    return result;
}
};

```

2.2. LazySegTree.h

```

class LazySegTree {
public:
    int n;
    vector<T> tree, lazy;
    // ---- in cases of assignment question jaha pe value change
    krni like assign
    // val p from [l,r] vector<bool> marked; // to track pending
    assignment

    LazySegTree(int size) : n(size) {
        tree.resize(4 * n + 1);
        lazy.resize(4 * n + 1, 0);
        // --- Uncomment when u need that assign val p in l,r vale
        // marked.resize(4 * n + 1, false);
    }

    void build(int node, int start, int end, const vector<T>
&arr) {
        if (start == end) {
            tree[node] = arr[start];
        } else {
            int mid = (start + end) / 2;
            build(2 * node + 1, start, mid, arr);
            build(2 * node + 2, mid + 1, end, arr);
            tree[node] = merge(tree[2 * node + 1], tree[2 * node +
2]);
        }
    }

    void apply_lazy(int node, int start, int end) {
        if (lazy[node] != 0) {
            tree[node] += (end - start + 1) * lazy[node];
            if (start != end) {
                lazy[2 * node + 1] += lazy[node];
                lazy[2 * node + 2] += lazy[node];
            }
        }
    }

```

```

    }
    lazy[node] = 0;
}

void update(int node, int start, int end, int l, int r, T
value) {
    apply_lazy(node, start, end);
    if (start > end || start > r || end < l)
        return;

    if (start >= l && end <= r) {
        tree[node] += (end - start + 1) * value;
        if (start != end) {
            lazy[2 * node + 1] += value;
            lazy[2 * node + 2] += value;
        }
        return;
    }

    int mid = (start + end) / 2;
    update(2 * node + 1, start, mid, l, r, value);
    update(2 * node + 2, mid + 1, end, l, r, value);
    tree[node] = merge(tree[2 * node + 1], tree[2 * node + 2]);
}

T query(int node, int start, int end, int l, int r) {
    apply_lazy(node, start, end);
    if (start > end || start > r || end < l)
        return identity();
    if (start >= l && end <= r)
        return tree[node];

    int mid = (start + end) / 2;
    return merge(query(2 * node + 1, start, mid, l, r),
        query(2 * node + 2, mid + 1, end, l, r));
}

```

```

    }

    //
    =====
    // ===== OPTIONAL MODE: RANGE ASSIGN
    =====
    // ===== (assignment vale question jusme purana add
nhi krna )
    // =====
    //
    =====

/*
void apply_lazy(int node, int start, int end) {
    if (marked[node]) {
        tree[node] = (end - start + 1) * lazy[node];
        if (start != end) {
            lazy[2 * node + 1] = lazy[node];
            lazy[2 * node + 2] = lazy[node];
            marked[2 * node + 1] = marked[2 * node + 2] =
true;
        }
        marked[node] = false;
    }
}

void update(int node, int start, int end, int l, int r, T
value) {
    apply_lazy(node, start, end);
    if (start > end || start > r || end < l) return;

    if (start >= l && end <= r) {
        tree[node] = (end - start + 1) * value;
        if (start != end) {
            lazy[2 * node + 1] = lazy[2 * node + 2] = value;
            marked[2 * node + 1] = marked[2 * node + 2] =

```

```

true;
        }
        return;
    }

    int mid = (start + end) / 2;
    update(2 * node + 1, start, mid, l, r, value);
    update(2 * node + 2, mid + 1, end, l, r, value);
    tree[node] = merge(tree[2 * node + 1], tree[2 * node +
2]);
}
*/

private:
    T merge(T a, T b) { return a + b; } // can replace with min/
max/gcd
    T identity() { return 0; } // replace as needed
};

```

2.3. LazySegTree2D.h

```

template <typename T>
class LazySegTree2D {
public:
    int n, m;
    vector<vector<T>> tree, lazy;
    // Uncomment these for assignment mode
    // vector<vector<bool>> marked;

    LazySegTree2D(int n_, int m_) : n(n_), m(m_) {
        tree.assign(4 * n, vector<T>(4 * m, 0));
        lazy.assign(4 * n, vector<T>(4 * m, 0));
        // marked.assign(4 * n, vector<bool>(4 * m, false));
    }

    // ===== BUILD =====

```

```

// here note we store vx and vy which are node of the seg
tree
// lx , rx are the range in x direction
// ly , ry are the range in y direction
void build_y(int vx, int lx, int rx, int vy, int ly, int ry,
             const vector<vector<T>>& a) {
    if (ly == ry) {
        if (lx == rx)
            tree[vx][vy] = a[lx][ly];
        else
            tree[vx][vy] = merge(tree[vx * 2][vy], tree[vx * 2 + 1]
[vy]);
    } else {
        int my = (ly + ry) / 2;
        build_y(vx, lx, rx, vy * 2, ly, my, a);
        build_y(vx, lx, rx, vy * 2 + 1, my + 1, ry, a);
        tree[vx][vy] = merge(tree[vx][vy * 2], tree[vx][vy * 2 +
1]);
    }
}

void build_x(int vx, int lx, int rx, const vector<vector<T>>&
a) {
    if (lx != rx) {
        int mx = (lx + rx) / 2;
        build_x(vx * 2, lx, mx, a);
        build_x(vx * 2 + 1, mx + 1, rx, a);
    }
    build_y(vx, lx, rx, 1, 0, m - 1, a);
}

// ===== LAZY APPLY (ADD MODE)
=====
void apply_lazy_y(int vx, int vy, int ly, int ry) {
    if (lazy[vx][vy] != 0) {
        tree[vx][vy] += (ry - ly + 1) * lazy[vx][vy];

```

```

    if (ly != ry) {
        lazy[vx][vy * 2] += lazy[vx][vy];
        lazy[vx][vy * 2 + 1] += lazy[vx][vy];
    }
    lazy[vx][vy] = 0;
}

// ===== UPDATE IN Y =====
void update_y(int vx, int lx, int rx,
              int vy, int ly, int ry,
              int y1, int y2, T val) {
    apply_lazy_y(vx, vy, ly, ry);
    if (ly > ry || ly > y2 || ry < y1) return;

    if (ly >= y1 && ry <= y2) {
        tree[vx][vy] += (ry - ly + 1) * val;
        if (ly != ry) {
            lazy[vx][vy * 2] += val;
            lazy[vx][vy * 2 + 1] += val;
        }
        return;
    }

    int my = (ly + ry) / 2;
    update_y(vx, lx, rx, vy * 2, ly, my, y1, y2, val);
    update_y(vx, lx, rx, vy * 2 + 1, my + 1, ry, y1, y2, val);
    tree[vx][vy] = merge(tree[vx][vy * 2], tree[vx][vy * 2 +
1]);
}

// ===== UPDATE IN X =====
void update_x(int vx, int lx, int rx,
              int x1, int x2, int y1, int y2, T val) {
    if (lx > rx || lx > x2 || rx < x1) return;

```

```

    if (lx >= x1 && rx <= x2) {
        update_y(vx, lx, rx, 1, 0, m - 1, y1, y2, val);
        return;
    }

    int mx = (lx + rx) / 2;
    update_x(vx * 2, lx, mx, x1, x2, y1, y2, val);
    update_x(vx * 2 + 1, mx + 1, rx, x1, x2, y1, y2, val);

    for (int vy = 1; vy < 4 * m; vy++) {
        tree[vx][vy] = merge(tree[vx * 2][vy], tree[vx * 2 + 1]
[vy]);
    }
}

// ===== QUERY IN Y =====
T query_y(int vx, int vy, int ly, int ry,
          int y1, int y2) {
    apply_lazy_y(vx, vy, ly, ry);
    if (ly > ry || ly > y2 || ry < y1) return identity();
    if (ly >= y1 && ry <= y2) return tree[vx][vy];

    int my = (ly + ry) / 2;
    return merge(
        query_y(vx, vy * 2, ly, my, y1, y2),
        query_y(vx, vy * 2 + 1, my + 1, ry, y1, y2));
}

// ===== QUERY IN X =====
T query_x(int vx, int lx, int rx,
          int x1, int x2, int y1, int y2) {
    if (lx > rx || lx > x2 || rx < x1) return identity();
    if (lx >= x1 && rx <= x2)
        return query_y(vx, 1, 0, m - 1, y1, y2);

    int mx = (lx + rx) / 2;

```

```

    T q1 = query_x(vx * 2, lx, mx, x1, x2, y1, y2);
    T q2 = query_x(vx * 2 + 1, mx + 1, rx, x1, x2, y1, y2);
    return merge(q1, q2);
}

private:
    T merge(T a, T b) { return a + b; } // can be min/max/gcd
    T identity() { return 0; }          // depends on merge op
};

```

2.4. Mos.h

```

int BLOCK = DO_NOT_FORGET_TO_CHANGE_THIS;
struct Query {
    int l, r, id;
    Query(int _l, int _r, int _id) : l(_l), r(_r), id(_id) {}
    bool operator<(Query &o) {
        int mblock = l / BLOCK, oblock = o.l / BLOCK;
        return (mblock < oblock) or
            (mblock == oblock and mblock % 2 == 0 and r < o.r)
or
            (mblock == oblock and mblock % 2 == 1 and r > o.r);
    };
};

void solve() {
    vector<Query> queries;
    queries.reserve(q);
    for (int i = 0; i < q; i++) {
        int l, r;
        cin >> l >> r;
        l--, r--;
        queries.emplace_back(l, r, i);
    }
    sort(all(queries));
    int ans = 0;
    auto add = [&](int v) {};
    auto rem = [&](int v) {};
}

```

```

vector<int> out(q); // Change out type if necessary
int cur_l = 0, cur_r = -1;
for (auto &[l, r, id] : queries) {
    while (cur_l > l)
        add(--cur_l);
    while (cur_l < l)
        rem(cur_l++);
    while (cur_r < r)
        add(++cur_r);
    while (cur_r > r)
        rem(cur_r--);
    out[id] = ans;
}
}

```

2.5. PersistentSegTree.h

```

struct Node {
    int val;
    Node *left, *right;
    Node(int v = 0, Node* l = nullptr, Node* r = nullptr)
        : val(v), left(l), right(r) {}
};

struct PersistentSegTree {
    int n;
    vector<Node*> version; // roots of each version

    PersistentSegTree(int n) : n(n) {
        version.push_back(build(0, n - 1)); // version[0]: initial
        (all 0)
    }

    Node* build(int l, int r) {
        if (l == r) return new Node(0);
        int m = (l + r) / 2;
        return new Node(0, build(l, m), build(m + 1, r));
    }
}

```

```

}

// create new version with arr[idx] += val
Node* update(Node* prev, int l, int r, int idx, int val) {
    if (l == r)
        return new Node(prev->val + val); // create new leaf
    node

    int m = (l + r) / 2;
    if (idx <= m)
        return new Node(prev->val + val,
                        update(prev->left, l, m, idx, val),
                        prev->right);
    else
        return new Node(prev->val + val,
                        prev->left,
                        update(prev->right, m + 1, r, idx, val));
}

int query(Node* node, int l, int r, int ql, int qr) {
    if (!node || ql > r || qr < l) return 0;
    if (ql <= l && r <= qr) return node->val;
    int m = (l + r) / 2;
    return query(node->left, l, m, ql, qr) +
           query(node->right, m + 1, r, ql, qr);
}

// user interface:
void update_version(int prev_version, int idx, int val) {
    Node* new_root = update(version[prev_version], 0, n - 1,
                             idx, val);
    version.push_back(new_root);
}

int query_version(int ver, int l, int r) {
    return query(version[ver], 0, n - 1, l, r);
}

```

```

    }
};

```

2.6. SegTree.h

```

template <typename T>
// Iterative segment tree with inclusive range queries
class segtree {
public:
    int n;
    vector<T> tree;

    segtree(int size) : n(size), tree(2 * n, identity()) {}

    segtree(const vector<T>& arr) : n((int)arr.size()), tree(2 *
n) {
        build(arr);
    }

    void build(const vector<T>& arr) {
        n = (int)arr.size();
        tree.assign(2 * n, identity());
        copy(arr.begin(), arr.end(), tree.begin() + n);
        for (int node = n - 1; node > 0; --node)
            tree[node] = merge(tree[node << 1], tree[node << 1 | 1]);
    }

    void update(int idx, T value) {
        for (tree[idx += n] = value; idx > 1; idx >>= 1)
            tree[idx >> 1] = merge(tree[idx], tree[idx ^ 1]);
    }

    T query(int l, int r) {
        T left_result = identity(), right_result = identity();
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) left_result = merge(left_result, tree[l++]);
            if (r & 1) right_result = merge(tree[--r], right_result);
        }
    }
};

```

```

    }
    return merge(left_result, right_result);
}

private:
    T merge(T a, T b) { return a + b; } // change to min/max/gcd
    T identity() { return 0; }          // change to the merge
identity
};

```

2.7. SegTree2D.h

```

template <typename T>
class SegTree2D {
public:
    int n, m;
    vector<vector<T>> tree;
    vector<vector<T>> arr;

    SegTree2D(int _n, int _m) : n(_n), m(_m) {
        arr.assign(n, vector<T>(m, 0));
        tree.assign(4 * n, vector<T>(4 * m, 0));
    }

    void build_y(int node_x, int start_x, int end_x, int node_y,
int start_y, int end_y) {
        if (start_y == end_y) {
            if (start_x == end_x)
                tree[node_x][node_y] = arr[start_x][start_y];
            else
                tree[node_x][node_y] = merge(
                    tree[node_x * 2][node_y],
                    tree[node_x * 2 + 1][node_y]);
        } else {
            int mid_y = (start_y + end_y) / 2;
            build_y(node_x, start_x, end_x, node_y * 2, start_y,
mid_y);
        }
    }
};

```

```

    build_y(node_x, start_x, end_x, node_y * 2 + 1, mid_y +
1, end_y);
    tree[node_x][node_y] = merge(
        tree[node_x][node_y * 2],
        tree[node_x][node_y * 2 + 1]);
}
}

void build_x(int node_x, int start_x, int end_x) {
    if (start_x != end_x) {
        int mid_x = (start_x + end_x) / 2;
        build_x(node_x * 2, start_x, mid_x);
        build_x(node_x * 2 + 1, mid_x + 1, end_x);
    }
    build_y(node_x, start_x, end_x, 1, 0, m - 1);
}

// Query on y-axis for a fixed x-segment
T query_y(int node_x, int node_y, int start_y, int end_y, int
y1, int y2) {
    if (y1 > end_y || y2 < start_y) return identity();
    if (y1 <= start_y && end_y <= y2) return tree[node_x]
[node_y];
    int mid_y = (start_y + end_y) / 2;
    return merge(
        query_y(node_x, node_y * 2, start_y, mid_y, y1, y2),
        query_y(node_x, node_y * 2 + 1, mid_y + 1, end_y, y1,
y2));
}

// Query on x-axis
T query_x(int node_x, int start_x, int end_x, int x1, int x2,
int y1, int y2) {
    if (x1 > end_x || x2 < start_x) return identity();
    if (x1 <= start_x && end_x <= x2)
        return query_y(node_x, 1, 0, m - 1, y1, y2);
}

```

```

    int mid_x = (start_x + end_x) / 2;
    return merge(
        query_x(node_x * 2, start_x, mid_x, x1, x2, y1, y2),
        query_x(node_x * 2 + 1, mid_x + 1, end_x, x1, x2, y1,
y2));
}

// Update on y-axis for fixed x-segment
void update_y(int node_x, int start_x, int end_x,
    int node_y, int start_y, int end_y,
    int x, int y, T val) {
    if (start_y == end_y) {
        if (start_x == end_x)
            tree[node_x][node_y] = val;
        else
            tree[node_x][node_y] = merge(
                tree[node_x * 2][node_y],
                tree[node_x * 2 + 1][node_y]);
    } else {
        int mid_y = (start_y + end_y) / 2;
        if (y <= mid_y)
            update_y(node_x, start_x, end_x, node_y * 2, start_y,
mid_y, x, y, val);
        else
            update_y(node_x, start_x, end_x, node_y * 2 + 1, mid_y
+ 1, end_y, x, y, val);

        tree[node_x][node_y] = merge(
            tree[node_x][node_y * 2],
            tree[node_x][node_y * 2 + 1]);
    }
}

// Update on x-axis
void update_x(int node_x, int start_x, int end_x, int x, int

```

```

y, T val) {
    if (start_x != end_x) {
        int mid_x = (start_x + end_x) / 2;
        if (x <= mid_x)
            update_x(node_x * 2, start_x, mid_x, x, y, val);
        else
            update_x(node_x * 2 + 1, mid_x + 1, end_x, x, y, val);
    }
    update_y(node_x, start_x, end_x, 1, 0, m - 1, x, y, val);
}

private:
T merge(T a, T b) { return a + b; } // use min/max/gcd as
needed
T identity() { return 0; }
};

```

2.8. range_update_tree.h

```

template <typename T, typename F>
struct RangeUpdateTree {
    int n;
    vector<T> tree;
    T identity;
    F merge;
    RangeUpdateTree(const vector<T> &arr, T id, F _m)
        : n((int)arr.size()), tree(2 * n), identity(id),
merge(_m) {
        for (int i = 0; i < n; i++)
            tree[n + i] = arr[i];
        for (int i = n - 1; i >= 1; i--)
            tree[i] = merge(tree[2 * i], tree[2 * i + 1]);
    }
    void update(int l, int r, T value) {
        assert(l >= 0 && r < n && l <= r);
        for (l += n, r += n; l <= r; l >>= 1, r >>= 1) {
            if (l & 1)

```

```

                tree[l] = merge(value, tree[l]), l++;
            if (!(r & 1))
                tree[r] = merge(value, tree[r]), r--;
        }
        if (l == r)
            tree[l] = merge(value, tree[l]);
    }
    T query(int v) {
        T res = tree[v += n];
        for (; v > 1; v >>= 1)
            res = merge(res, tree[v >> 1]);
        return res;
    }
};
// ex: RangeUpdateTree<int, decltype(join)> v(vi (n, 1e9), 1e9,
join); use auto
// func for join

```

3. Graph

3.1. HLD.h

```

struct HLD {
    int n, timer = 0;
    vi top, tin, p, sub;
    HLD(vvi &adj) : n(sz(adj)), top(n), tin(n), p(n, -1), sub(n,
1) {
        vi ord(n + 1);
        for (int i = 0, t = 0, v = ord[i]; i < n; v = ord[++i])
            for (auto &to : adj[v])
                if (to != p[v])
                    p[to] = v, ord[++t] = to;
        for (int i = n - 1, v = ord[i]; i > 0; v = ord[--i])
            sub[p[v]] += sub[v];
        for (int v = 0; v < n; v++)
            if (sz(adj[v]))
                iter_swap(begin(adj[v]), max_element(all(adj[v]), [&

```

```

(int a, int b) {
    return make_pair(a != p[v], sub[a]) <
           make_pair(b != p[v], sub[b]);
});
function<void(int)> dfs = [&](int v) {
    tin[v] = timer++;
    for (auto &to : adj[v])
        if (to != p[v]) {
            top[to] = (to == adj[v][0] ? top[v] : to);
            dfs(to);
        }
};
dfs(0);
}
int lca(int u, int v) {
    return process(u, v, [](int, int) {});
}
template <class B>
int process(int a, int b, B op, bool ignore_lca = false) {
    for (int v;; op(tin[v], tin[b]), b = p[v]) {
        if (tin[a] > tin[b])
            swap(a, b);
        if ((v = top[b]) == top[a])
            break;
    }
    if (int l = tin[a] + ignore_lca, r = tin[b]; l <= r)
        op(l, r);
    return a;
}
template <class B>
void subtree(int v, B op, bool ignore_lca = false) {
    if (sub[v] > 1 or !ignore_lca)
        op(tin[v] + ignore_lca, tin[v] + sub[v] - 1);
}
};

```

3.2. KthAnc.h

// O(log n) LCA with Kth anc

```

struct LCA {
    int n;
    vvi &adjLists;
    int lg;
    vvi up;
    vi depth;
    LCA(vvi &adjLists, int root = 0) : n(sz(_adjLists)),
    adjLists(_adjLists) {
        lg = 1;
        int pw = 1;
        while (pw <= n)
            pw <= 1, lg++;
        // lg = 20
        up = vvi(n, vi(lg));
        depth.assign(n, -1);
        function<void(int, int)> parentDFS = [&](int from, int
parent) {
            depth[from] = depth[parent] + 1;
            up[from][0] = parent;
            for (auto to : adjLists[from]) {
                if (to == parent)
                    continue;
                parentDFS(to, from);
            }
        };
        parentDFS(root, root);
        for (int j = 1; j < lg; j++) {
            for (int i = 0; i < n; i++) {
                up[i][j] = up[up[i][j - 1]][j - 1];
            }
        }
    }
    int kthAnc(int v, int k) {
        int ret = v;

```

```

    int pw = 0;
    while (k) {
        if (k & 1)
            ret = up[ret][pw];
        k >>= 1;
        pw++;
    }
    return ret;
}
int lca(int u, int v) {
    if (depth[u] > depth[v])
        swap(u, v);
    v = kthAnc(v, depth[v] - depth[u]);
    if (u == v)
        return v;
    while (up[u][0] != up[v][0]) {
        int i = 0;
        for (; i < lg - 1; i++) {
            if (up[u][i + 1] == up[v][i + 1])
                break;
        }
        u = up[u][i], v = up[v][i];
    }
    return up[u][0];
};
int dist(int u, int v) { return depth[u] + depth[v] - 2 *
depth[lca(u, v)]; }
};

```

3.3. LCA.h

```

// O(1) LCA
struct LCA {
    int T = 0;
    vi st, path, ret;
    vi en, d;
    RMQ<int> rmq;

```

```

    LCA(vector<vi> &C)
        : st(sz(C)), en(sz(C)), d(sz(C)), rmq((dfs(C, 0, -1),
ret)) {}
    void dfs(vvi &adj, int v, int par) {
        st[v] = T++;
        for (auto to : adj[v])
            if (to != par) {
                path.pb(v), ret.pb(st[v]);
                d[to] = d[v] + 1;
                dfs(adj, to, v);
            }
        en[v] = T - 1;
    }
    bool anc(int p, int c) { return st[p] <= st[c] and en[p] >=
en[c]; }
    int lca(int a, int b) {
        if (a == b)
            return a;
        tie(a, b) = minmax(st[a], st[b]);
        return path[rmq.query(a, b - 1)];
    }
    int dist(int a, int b) { return d[a] + d[b] - 2 * d[lca(a,
b)]; }
};

```

3.4. bellman.h

```

struct Edge {
    int a, b, cost;
};

int n, m, v;
vector<Edge> edges;
const int INF = 1000000000;

void solve() {
    vector<int> d(n, INF);

```

```

d[v] = 0;
vector<int> p(n, -1);
int x;
for (int i = 0; i < n; ++i) {
    x = -1;
    for (Edge e : edges)
        if (d[e.a] < INF)
            if (d[e.b] > d[e.a] + e.cost) {
                d[e.b] = max(-INF, d[e.a] + e.cost);
                p[e.b] = e.a;
                x = e.b;
            }
}

if (x == -1)
    cout << "No negative cycle from " << v;
else {
    int y = x;
    for (int i = 0; i < n; ++i)
        y = p[y];

    vector<int> path;
    for (int cur = y;; cur = p[cur]) {
        path.push_back(cur);
        if (cur == y && path.size() > 1)
            break;
    }
    reverse(path.begin(), path.end());

    cout << "Negative cycle: ";
    for (int u : path)
        cout << u << ' ';
}
}

```

3.5. bridges.h

```

int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    for (int to : adj[v]) {
        if (to == p)
            continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}

```

```
// ARTICULATION POINTS:
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> tin, low;
int timer;
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;
    for (int to : adj[v]) {
        if (to == p)
            continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}
```

```
// arya bridges
void findBridges_dfs(int u, int p, int &time,
vector<vector<int>> &adj,
vector<int> &disc, vector<int> &low,
vector<pair<int, int>> &bridges) {
    disc[u] = low[u] = time++;

    for (int v : adj[u]) {
        if (v == p)
            continue;

        if (disc[v] != -1) {
            low[u] = min(low[u], disc[v]);
        } else {
            findBridges_dfs(v, u, time, adj, disc, low, bridges);
            low[u] = min(low[u], low[v]);
            if (low[v] > disc[u]) {
                bridges.push_back({u, v});
            }
        }
    }
}

vector<pair<int, int>> findBridges(int n, vector<vector<int>>
&adj) {
    vector<int> disc(n, -1), low(n, -1);
    vector<pair<int, int>> bridges;
    int time = 0;

    for (int i = 0; i < n; ++i) {
        if (disc[i] == -1) {
            findBridges_dfs(i, -1, time, adj, disc, low, bridges);
        }
    }
}
```

```
    return bridges;
}
```

3.6. dijkstra.h

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int>& d, vector<int>& p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    set<pair<int, int>> q;
    q.insert({0, s});
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase(q.begin());

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                q.erase({d[to], to});
                d[to] = d[v] + len;
                p[to] = v;
                q.insert({d[to], to});
            }
        }
    }
}
```

3.7. dsu.h

```
struct DSU {
    vector<int> p, sz;
```

```
DSU(int n) {
    p.resize(n);
    sz.assign(n, 1);
    for (int i = 0; i < n; ++i) p[i] = i;
}

int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
bool unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) return false;
    if (sz[a] < sz[b]) swap(a, b);
    p[b] = a;
    sz[a] += sz[b];
    return true;
}

int size(int x) { return sz[find(x)]; }
};
```

3.8. floyd_washall.h

```
const long long INF = (long long)1e18;
bool floyd_warshall(int n, vector<vector<long long>> &dist) {
    // initialise dist with edge weights, INF if no edge exists
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] < INF && dist[k][j] < INF)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k]
[j]);

    for (int i = 0; i < n; i++)
        if (dist[i][i] < 0)
            return true;
    return false;
}
```

4. Number Theory

4.1. ModularArithmetic.h

```
const long long MOD = 1e9 + 7;

long long mod_mul(long long a, long long b, long long mod = MOD) {
    a = a % mod;
    b = b % mod;
    return ((a * b) % mod + mod) % mod;
}

long long mod_add(long long a, long long b, long long mod = MOD) {
    a = a % mod;
    b = b % mod;
    return ((a + b) % mod + mod) % mod;
}

int modexp(int base, int exp, int mod = MOD) {
    int res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) res = mod_mul(res, base, mod);
        base = mod_mul(base, base, mod);
        exp >>= 1;
    }
    return res;
}

int inv(int a, int m = MOD) {
    return modexp(a, m - 2, m);
}

vi fact(int n, int mod = MOD) {
    vi f(n + 1, 1);
```

```
    for (int i = 2; i <= n; i++)
        f[i] = mod_mul(f[i - 1], i, mod);
    return f;
}

vi invfact(int n, int mod = MOD) {
    vi facts = fact(n, mod);
    vi invf(n + 1, 1);
    invf[n] = modexp(facts[n], mod - 2, mod);
    for (int i = n - 1; i >= 0; i--)
        invf[i] = mod_mul(invf[i + 1], i + 1, mod);
    return invf;
}

vector<int> comb_fact, comb_invfact;

void init_comb(int n, int mod = MOD) {
    comb_fact = fact(n, mod);
    comb_invfact = invfact(n, mod);
}

int nCr(int n, int r, int mod = MOD) {
    if (r < 0 || r > n) return 0;
    int res = mod_mul(comb_fact[n], comb_invfact[r], mod);
    return mod_mul(res, comb_invfact[n - r], mod);
}

4.2. bit_bns.h

// --- Bit Binary Search in o(log(n)) ---
const int M = 20 const int N = 1 << M

int
lower_bound(int val) {

    int ans = 0, sum = 0;
    for (int i = M - 1; i >= 0; i--) {
        int x = ans + (1 << i);
```

```

    if (sum + bit[x] < val)
        ans = x, sum += bit[x];
}

return ans + 1;
}

```

4.3. knapsack.h

```

for (int i = 1; i <= n; i++)
    for (int j = W; j >= w[i]; j--)
        f[j] = max(f[j], f[j - w[i]] + v[i]);

```

4.4. matrix_expo.h

```

int **matrixmul(int **matrix1, int **matrix2) {
    int **matrix3 = new int *[2];
    for (int i = 0; i < 2; i++)
        matrix3[i] = new int[2];

    matrix3[0][0] =
        (matrix1[0][0] * matrix2[0][0]) + (matrix1[0][1] *
matrix2[1][0]);
    matrix3[0][1] =
        (matrix1[0][0] * matrix2[0][1]) + (matrix1[0][1] *
matrix2[1][1]);
    matrix3[1][0] =
        (matrix1[1][0] * matrix2[0][0]) + (matrix1[1][1] *
matrix2[1][0]);
    matrix3[1][1] =
        (matrix1[1][0] * matrix2[0][1]) + (matrix1[1][1] *
matrix2[1][1]);
    matrix3[0][0] %= M;
    matrix3[1][0] %= M;
    matrix3[0][1] %= M;
    matrix3[1][1] %= M;

    return matrix3;
}

```

```

}

int **matrixexpo(int **matrix, int n, int **ans) {
    while (n > 0) {
        if (n % 2 == 1)
            ans = matrixmul(ans, matrix);
        matrix = matrixmul(matrix, matrix);
        n /= 2;
    }
    return ans;
}

```

4.5. my_math.h

```

using u64 = uint64_t;
using i64 = int64_t;

u64 mult(u64 a, u64 b, u64 m = MOD) {
    // performs modular multiplication (a*b)%m avoiding overflow
    i64 ret = a * b - m * (u64)(1.L / m * a * b);
    return ret + m * (ret < 0) - m * (ret >= (i64)m);
}

u64 pw(u64 b, u64 e, u64 m = MOD) {
    // performs modular exponentiation (b^e)%m avoiding overflow
    using the above
    // mult function u can use ur bexp fnc as well
    u64 ret = 1;
    for (; e; b = mult(b, b, m), e >>= 1)
        if (e & 1)
            ret = mult(ret, b, m);
    return ret;
}

bool isPrime(u64 n) {
    // miller rabin primality test for 64 bit integers
    if (n < 2 || n % 6 % 4 != 1)

```

```

    return (n | 1) == 3;
u64 A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
    s = __builtin_ctzll(n - 1), d = n >> s;
for (u64 a : A) {
    // note we are reducing a mod n as a might be larger than n
    // so it is correct
    // as per the condn of less than n-1
    u64 p = pw(a % n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
        p = mult(p, p, n);
    if (p != n - 1 && i != s)
        return 0;
}
return 1;
}

```

```

using ull = u64;
ull pollard(ull n) {
    // pollard's rho algo which is probabilistic but works well
    // in practice this
    // will return a nontrivial factor of n
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return mult(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y)
            x = ++i, y = f(x);
        if ((q = mult(prd, max(x, y) - min(x, y), n)))
            prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ull> factor(ull n) {
    // returns the prime factorization of n in  $O(n^{0.25})$  time
    if (n == 1)
        return {};
}

```

```

if (isPrime(n))
    return {n};
ull x = pollard(n);
auto l = factor(x), r = factor(n / x);
l.insert(l.end(), all(r));
return l;
}

```

4.6. some_dp.h

```

// LIS
int lis(vector<int> const &a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> d(n + 1, INF);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
        if (d[l - 1] < a[i] && a[i] < d[l])
            d[l] = a[i];
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (d[l] < INF)
            ans = l;
    }
    return ans;
}
// or segtree lol

```

4.7. spf.h

```

int MX = 1e7 + 1;
vi spf(MX + 1, INT32_MAX);
vector<int> is_prime(MX + 1, 1);
void sieve(int n = MX) {
}

```

```

is_prime[0] = is_prime[1] = 0;
int cnt = 1;
for (int i = 2; i <= n; i++) {
    if (is_prime[i]) {
        for (int j = i * i; j <= n; j += i) {
            is_prime[j] = 0;
            spf[j] = min(i, spf[j]);
        }
        is_prime[i] = cnt;
        cnt++;
    }
}
return;
}

```

5. Strings

5.1. Trie.h

```

class TrieNode {
public:
    unordered_map<char, TrieNode *> children;
    bool isEndOfWord;

    TrieNode() : isEndOfWord(false) {}
};

class Trie {
private:
    TrieNode *root;

public:
    Trie() { root = new TrieNode(); }
    void insert(const string &word) {
        TrieNode *node = root;
        for (char ch : word) {
            if (node->children.find(ch) == node->children.end()) {
                node->children[ch] = new TrieNode();
            }
            node = node->children[ch];
        }
        node->isEndOfWord = true;
    }
    bool search(const string &word) {
        TrieNode *node = root;
        for (char ch : word) {
            if (node->children.find(ch) == node->children.end()) {
                return false;
            }
            node = node->children[ch];
        }
        return node->isEndOfWord;
    }
    bool startsWith(const string &prefix) {
        TrieNode *node = root;
        for (char ch : prefix) {
            if (node->children.find(ch) == node->children.end()) {
                return false;
            }
            node = node->children[ch];
        }
        return true;
    }
};

```

```

    }
    node = node->children[ch];
}
node->isEndOfWord = true;
}
bool search(const string &word) {
    TrieNode *node = root;
    for (char ch : word) {
        if (node->children.find(ch) == node->children.end()) {
            return false;
        }
        node = node->children[ch];
    }
    return node->isEndOfWord;
}
bool startsWith(const string &prefix) {
    TrieNode *node = root;
    for (char ch : prefix) {
        if (node->children.find(ch) == node->children.end()) {
            return false;
        }
        node = node->children[ch];
    }
    return true;
}
};

```

5.2. hash.h

```

template <int MOD, int P>
struct RH {
    // using H1 = RH<1000000007, 91138233>;
    // using H2 = RH<1000000009, 97266353>;
    vector<long long> h, p;
    RH(const string &s) {
        int n = s.size();
        h.resize(n + 1, 0);
    }
};

```

```

    p.resize(n + 1, 0);
    p[0] = 1;
    for (int i = 0; i < n; i++) {
        h[i + 1] = (h[i] * P + s[i]) % MOD;
        p[i + 1] = p[i] * P % MOD;
    }
}
long long get(int l, int r) { // [l, r]
    long long res = (h[r + 1] - h[l] * p[r - l + 1]) % MOD;
    return res < 0 ? res + MOD : res;
}
};

```

5.3. kmp.h

```

vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> KMP(string text, string pattern) {
    string s = pattern + "#" + text;
    vector<int> pi = prefix_function(s);
    vector<int> matches;
    int p = pattern.length();

    for (int i = 0; i < s.length(); i++) {
        if (pi[i] == p) {

```

```

            int match_pos = i - 2 * p;
            matches.push_back(match_pos); // 0-based index in text
        }
    }
    return matches;
}

```

6. Theory

6.1. Combinatorics

Binomial Theorem:

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k$$

Expands powers of a binomial.

Combinations (nC_r):

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

Number of ways to choose r items from n distinct items without replacement, order does not matter.

Permutations (nP_r):

$$P(n, r) = \frac{n!}{(n-r)!}$$

Number of ways to arrange r items from n distinct items, order matters.

Stars and Bars: Used to find the number of integer solutions to $x_1 + x_2 + \dots + x_n = k$.

- Case $x_i \geq 0$ (non-negative integers):

$$\binom{n+k-1}{n-1}$$

or

$$\binom{n+k-1}{k}$$

- Case $x_i \geq 1$ (positive integers):

$$\binom{k-1}{n-1}$$

Catalan Numbers (C_n):

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

- Number of valid parenthesis sequences of length $2n$.
- Number of structurally unique binary trees with n nodes is C_n .
- Number of rooted ordered trees with n nodes is $C_{\{n-1\}}$.

Derangements: Permutations where no element appears in its original position.

- Recurrence:

$$D_n = (n-1)(D_{\{n-1\}} + D_{\{n-2\}})$$

with $D_1 = 0, D_2 = 1$.

- General Formula:

$$D_n = n! \sum_{i=0}^n \frac{(-1)^i}{i!}$$

Principle of Inclusion-Exclusion (PIE): Used to count the number of elements in the union of overlapping sets.

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

6.2. Geometry

Pick's Theorem: Finds the area of a polygon whose vertices are all on integer grid points.

$$\text{Area} = I + \frac{B}{2} - 1$$

Where I is the number of interior grid points and B is the number of boundary grid points.